

UNIVERSIDAD RAFAEL LANDÍVAR
FACULTAD DE INGENIERÍA
ARQUITECTURA DE DATOS
MSC. LUIS RICARDO SIERRA

Proyecto Final

Nombre	Carné
DIEGO JAVIER MORALES MONZÓN	1132119
ESTUARDO JOSÉ VILLEDA NAVARRO	1003519
JORGE WALDEMAR GARCÍA BALDIZÓN	1220019
JOSÉ DANIEL DE LEÓN CHANG	1170419

Guatemala de la Asunción, 1 de mayo del 2026

Resumen Ejecutivo

ACME EV es una empresa dedicada a la comercialización de vehículos eléctricos que busca fortalecer su estrategia mediante el uso inteligente de los datos. Su flota genera información GPS y datos de estado operativo de forma continua, lo que representa una oportunidad para mejorar la experiencia del cliente, optimizar la operación de las sucursales y habilitar capacidades de analítica para la movilidad eléctrica.

El proyecto propone una arquitectura de datos moderna, escalable y segura basada en el enfoque Kappa. Esta arquitectura utiliza un flujo centralizado de eventos para atender procesamiento en tiempo real y análisis histórico mediante el reprocesamiento de datos almacenados, evitando duplicidad de lógica entre rutas batch y streaming.

La solución contempla Apache Kafka para ingesta, Spark Structured Streaming para procesamiento, PostgreSQL para datos estructurados, MongoDB para datos semiestructurados y un Data Lake para conservar datos en formatos JSON, CSV y Parquet. Además, incorpora seguridad, gobernanza, calidad y trazabilidad para proteger información sensible y apoyar la toma de decisiones, mantenimiento predictivo y servicios digitales para clientes.

Introducción

La movilidad eléctrica se ha convertido en un sector estratégico dentro de la transformación digital. Las empresas que comercializan vehículos eléctricos no solo compiten por la calidad de sus productos, sino también por su capacidad de ofrecer servicios inteligentes basados en datos. En este contexto, la información generada por los vehículos se convierte en un activo clave para mejorar la experiencia del cliente, optimizar procesos internos y generar nuevas oportunidades de negocio.

ACME EV enfrenta el reto de administrar datos GPS generados cada treinta segundos y datos de estado operativo generados cada minuto. Por ello, este informe propone una arquitectura de datos basada en Kappa, capaz de recibir, procesar, almacenar y consultar información en tiempo real, manteniendo la posibilidad de realizar análisis histórico mediante reprocesamiento de eventos almacenados.

Descripción del Caso de Negocio

ACME EV es una empresa dedicada a la venta de vehículos eléctricos con presencia en diez sucursales distribuidas en distintos países. Actualmente cuenta con aproximadamente diez mil vehículos en circulación y proyecta un crecimiento anual del veinte por ciento, lo que incrementará de forma constante el volumen de datos generado por la flota.

Cada vehículo transmite datos GPS cada treinta segundos, incluyendo identificador del vehículo, fecha y hora, latitud y longitud. Estos datos deben estar disponibles durante treinta días para consulta y descarga por parte de los propietarios. Además, los vehículos transmiten datos de estado operativo cada minuto, como batería, encendido, código de problema y kilometraje, los cuales deben conservarse durante trescientos sesenta y cinco días para análisis de fallas, mantenimiento y operación de sucursales.

Un aspecto relevante es que futuros modelos de vehículos enviarán información adicional de estado con estructuras aún no definidas. Por ello, la arquitectura debe ser flexible, escalable y capaz de evolucionar sin afectar la operación actual.

Objetivos del Proyecto

Objetivo General

Diseñar una arquitectura de datos moderna, escalable, segura y eficiente que permita a ACME EV gestionar, procesar y analizar datos generados por su flota de vehículos eléctricos, tanto en tiempo real como de forma histórica, con el propósito de mejorar la toma de decisiones, optimizar la operación del negocio y ofrecer servicios de valor agregado a sus clientes.

Objetivos Específicos

- Diseñar una arquitectura de datos basada en el enfoque Kappa, centrada en eventos, que integre ingesta, procesamiento, almacenamiento y consulta de datos vehiculares.
- Definir una estrategia de ingesta continua capaz de recibir datos GPS y datos de estado generados por múltiples vehículos de forma simultánea.
- Organizar los eventos vehiculares en canales o tópicos diferenciados para ubicación GPS y estado operativo del vehículo.
- Establecer mecanismos de almacenamiento y retención que conserven datos GPS durante treinta días y datos de estado durante trescientos sesenta y cinco días.
- Habilitar el análisis histórico mediante el almacenamiento y reprocesamiento de eventos, evitando duplicar procesos entre cargas batch y procesamiento en tiempo real.
- Definir mecanismos de acceso seguro para que clientes, sucursales y áreas internas consulten únicamente la información que les corresponde.
- Incorporar controles de seguridad, gobernanza, calidad, linaje, auditoría y trazabilidad de datos.
- Proponer un prototipo funcional con datos sintéticos que demuestre la generación, ingesta, procesamiento, almacenamiento y consulta de datos GPS y de estado vehicular.

Análisis de la Solución

La solución propuesta debe responder a tres necesidades principales: operación en tiempo real, análisis histórico y acceso controlado a la información. La operación en tiempo real es necesaria para detectar eventos relevantes, como fallas reportadas por los vehículos, niveles bajos de batería o cambios de estado. El análisis histórico es necesario para identificar patrones de uso, tendencias, fallas recurrentes y oportunidades de mantenimiento predictivo. El acceso controlado es indispensable para garantizar que cada usuario consulte únicamente la información que le corresponde.

Para cubrir estas necesidades, se propone una arquitectura Kappa. Los eventos generados por los vehículos ingresan a Apache Kafka y se procesan mediante un pipeline central de streaming. El análisis histórico se atiende reprocesando eventos desde Kafka, cuando la retención lo permita, o desde el Data Lake, sin mantener una lógica batch separada para las mismas reglas de negocio.

El enfoque Kappa también permite cumplir con las políticas de retención diferenciadas. Los datos GPS, al tener alta frecuencia y vigencia corta, pueden almacenarse con mecanismos optimizados para consultas recientes y eliminación automática después de treinta días. Los datos de estado, al tener mayor valor para mantenimiento y análisis operativo, se conservan durante trescientos sesenta y cinco días y pueden reutilizarse para reprocesamiento histórico.

Diseño Conceptual de la Arquitectura

El diseño conceptual describe los componentes principales de la solución desde una perspectiva de negocio. La arquitectura inicia con los vehículos eléctricos, que actúan como fuentes generadoras de datos. Cada vehículo transmite eventos GPS y eventos de estado hacia una capa central de ingesta.

La capa de ingesta recibe los eventos de forma continua y los organiza en flujos separados según su tipo: datos GPS y datos de estado. Esta separación permite aplicar reglas distintas de almacenamiento, procesamiento, retención y consulta.

Después de la ingesta, los datos se procesan mediante un flujo centralizado de eventos. El mismo pipeline valida, transforma y distribuye los datos para consultas operativas, alertas, almacenamiento histórico y consumo analítico. Cuando se requieren reportes acumulados o recalcular indicadores, el sistema reprocessa eventos almacenados en lugar de utilizar una ruta batch independiente.

La información procesada se almacena en diferentes repositorios según su propósito. Los datos operativos estructurados pueden almacenarse en una base de datos relacional. Los datos semiestructurados o con esquema variable pueden almacenarse en una base NoSQL. Los datos históricos y archivos analíticos pueden conservarse en un Data Lake usando formatos eficientes como Parquet.

La arquitectura expone servicios de consulta para dos grupos principales. Los propietarios de vehículos pueden consultar y descargar sus datos GPS, mientras que las sucursales pueden consultar datos de estado para análisis de mantenimiento, fallas y operación.

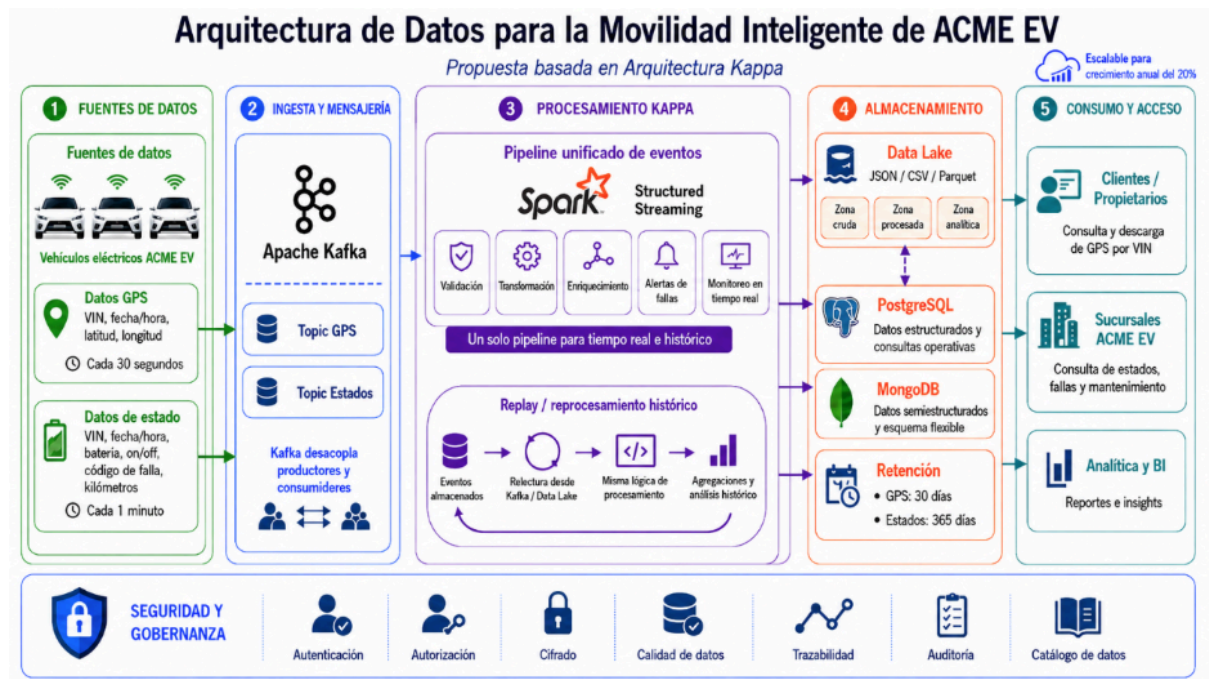


Figura 1. Arquitectura Kappa propuesta para ACME EV.

Justificación de la Arquitectura Kappa

La arquitectura Kappa fue seleccionada porque el caso de ACME EV se basa en flujos continuos de datos generados por vehículos conectados. Los eventos GPS y de estado tienen valor inmediato para operación y monitoreo, pero también conservan valor histórico para análisis, reportes y mantenimiento predictivo.

El procesamiento en tiempo real permite atender eventos recientes con baja latencia. Esto resulta útil para detectar códigos de falla, cambios en el estado del vehículo, niveles críticos de batería o condiciones que requieran atención rápida por parte de una sucursal.

El análisis histórico no requiere una capa batch separada. En el enfoque Kappa, los eventos almacenados en Kafka o en el Data Lake pueden reprocesarse utilizando la misma lógica de validación, transformación y enriquecimiento aplicada al flujo en tiempo real.

Esta decisión reduce la duplicidad de procesos, evita inconsistencias entre resultados batch y streaming, simplifica el mantenimiento y permite que los cambios en reglas de negocio se implementen en un único pipeline. Además, facilita la comparación entre procesamiento en tiempo real y reprocesamiento histórico dentro del prototipo.

Diseño de Ingeniería

El diseño de ingeniería define cómo se implementaría técnicamente la arquitectura propuesta. Para el prototipo se recomienda una implementación local basada en herramientas open source, utilizando Docker para facilitar la instalación y ejecución de los componentes.

La generación de datos sintéticos puede realizarse mediante un backend desarrollado en Node.js. Este servicio simulará vehículos enviando eventos GPS y eventos de estado de forma periódica. Para el prototipo, los eventos pueden generarse cada diez segundos, manteniendo correlación geográfica para evitar desplazamientos irreales. Los mensajes se serializarán en formato JSON por su flexibilidad para representar estructuras variables.

Apache Kafka será utilizado como plataforma de mensajería para recibir los eventos generados por los vehículos. Se propone crear al menos dos tópicos principales: uno para datos GPS y otro para datos de estado. Esta separación facilita el procesamiento independiente de cada flujo.

Apache Spark Structured Streaming será utilizado para procesar los datos. Spark consumirá eventos desde Kafka para aplicar validaciones, transformaciones y reglas de negocio de forma continua. Para necesidades históricas, Spark podrá leer eventos almacenados en Kafka o en el Data Lake y ejecutar la misma lógica de procesamiento para recalcular indicadores, generar agregados o corregir información.

PostgreSQL se utilizará como base de datos relacional para almacenar datos estructurados que requieran consultas transaccionales o controladas, como información de vehículos, propietarios, sucursales y consultas operativas recientes. MongoDB se utilizará como base NoSQL para almacenar datos de estado con posibles atributos variables, considerando que futuros modelos de vehículos podrían enviar estructuras diferentes.

El Data Lake almacenará archivos crudos y procesados en formatos JSON, CSV y Parquet. JSON será útil para datos crudos, CSV para intercambios simples y Parquet para almacenamiento analítico eficiente. Además, el Data Lake funcionará como repositorio histórico para auditoría, trazabilidad y reprocesamiento de eventos bajo el enfoque Kappa.

Tecnologías Propuestas

Componente	Tecnología propuesta	Justificación
Generación de datos	Node.js	Permite simular vehículos, generar datos sintéticos correlacionados y exponer APIs REST para consultas del prototipo.
Ingesta de eventos	Apache Kafka	Soporta flujos continuos, desacopla productores y consumidores, alimenta tópicos y permite integración con conectores.
Procesamiento streaming	Apache Spark Structured Streaming	Permite procesar eventos en tiempo casi real y aplicar validaciones y transformaciones distribuidas.
Reprocesamiento histórico	Apache Spark Structured Streaming / Spark	Permite reprocesar eventos almacenados en Kafka o en el Data Lake utilizando la misma lógica del flujo principal.
Base relacional	PostgreSQL	Adecuada para datos estructurados, consultas SQL y control de integridad.
Base NoSQL	MongoDB	Flexible para datos semiestructurados y evolución de esquemas.
Data Lake local	MinIO o sistema de archivos local	Permite almacenar archivos crudos y procesados en distintos formatos.
Formatos de datos	JSON, CSV, Parquet	Permiten demostrar serialización flexible, intercambio simple y eficiencia analítica.
Orquestación local	Docker Compose	Facilita levantar el ambiente completo de forma reproducible.
Visualización opcional	Power BI	Permite crear dashboards para análisis y presentación.

Seguridad y Gobernanza de Datos

La arquitectura debe proteger la información sensible generada por los vehículos, especialmente los datos de ubicación GPS. Para ello, se implementarán controles de autenticación, autorización, cifrado, trazabilidad, auditoría y retención de datos.

La autenticación permitirá identificar a los usuarios que acceden al sistema, mientras que la autorización determinará la información disponible para cada perfil. Bajo este modelo, un

propietario únicamente podrá consultar los datos GPS de sus vehículos, mientras que una sucursal tendrá acceso a la información operativa de las unidades bajo su responsabilidad.

En el clúster de Kafka, los eventos GPS y de estado se conservarán temporalmente en Kafka durante siete días para permitir su recuperación o reprocesamiento; el almacenamiento histórico definitivo se realizará en PostgreSQL, MongoDB y el Data Lake.

Los datos en tránsito se protegerán mediante protocolos seguros, como HTTPS y TLS, y la información sensible almacenada deberá contar con mecanismos de cifrado en reposo. Cuando los datos se utilicen para análisis agregados o modelos predictivos, se aplicarán técnicas de enmascaramiento o anonimización para reducir el riesgo de exposición.

La gobernanza de datos incluirá la asignación de responsables, un catálogo de datos, reglas de calidad, linaje, auditoría y políticas de retención. Los datos GPS se conservarán durante treinta días, mientras que los datos de estado vehicular se mantendrán durante trescientos sesenta y cinco días. Una vez cumplidos estos períodos, la información deberá eliminarse o archivararse de acuerdo con las necesidades del negocio.

Política de backups: Se realizarán respaldos incrementales cada hora para limitar la pérdida de información sin generar una carga excesiva sobre las bases de datos.

RTO: 24 horas como tiempo máximo para restablecer los servicios, considerando que la información se utiliza principalmente para mantenimiento preventivo y análisis operativo.

RPO: 1 hora como pérdida máxima aceptable de datos, en concordancia con la frecuencia establecida para los respaldos incrementales.

Plan de Escalabilidad a Cinco Años

La flota actual de ACME EV es de aproximadamente diez mil vehículos y se estima un crecimiento anual del veinte por ciento. Bajo esta proyección, la cantidad de vehículos crecería de forma significativa durante los próximos cinco años, incrementando proporcionalmente el volumen de eventos GPS y de estado.

Para soportar este crecimiento, la arquitectura debe escalar horizontalmente. Apache Kafka puede aumentar su capacidad mediante particiones y nuevos brokers. Apache Spark Structured Streaming puede incrementar su capacidad agregando nodos de procesamiento. Las bases de datos pueden optimizarse mediante particionamiento, índices y réplicas de lectura. El enfoque Kappa favorece la escalabilidad porque concentra la lógica de procesamiento en un flujo principal de eventos.

También se recomienda separar los datos por zonas: zona cruda, zona procesada y zona analítica. Esta organización facilita el mantenimiento, la trazabilidad y la reutilización de los datos. La zona cruda conserva los eventos originales, la zona procesada almacena datos limpios y validados, y la zona analítica contiene información preparada para reportes, indicadores y reprocesamiento histórico.

El escalamiento debe acompañarse de monitoreo continuo. Se deben medir indicadores como volumen de eventos por segundo, latencia de procesamiento, uso de almacenamiento, tiempos de consulta, errores de ingesta y disponibilidad de servicios.

Prototipo Propuesto

El prototipo tiene como objetivo demostrar de forma práctica el flujo completo de datos desde la generación de eventos hasta la consulta por parte de clientes y sucursales.

La primera parte consistirá en un generador de datos sintéticos desarrollado en Node.js. Este generador simulará vehículos enviando eventos GPS y eventos de estado. Cada evento GPS incluirá VIN, fecha y hora, latitud y longitud. Cada evento de estado incluirá identificador del dispositivo, fecha y hora, porcentaje de batería, estado de encendido, código de problema y kilometraje.

La segunda parte consistirá en enviar estos eventos a Apache Kafka. Se crearán tópicos separados para GPS y estados, permitiendo simular el comportamiento de una plataforma de ingesta en tiempo real.

La tercera parte consistirá en consumir los eventos mediante un proceso de streaming. Los datos GPS podrán almacenarse en PostgreSQL o en archivos Parquet con retención de treinta días. Los datos de estado podrán almacenarse en MongoDB y en el Data Lake con retención de trescientos sesenta y cinco días. Los eventos históricos podrán reprocesarse desde el Data Lake para demostrar el enfoque Kappa.

La cuarta parte consistirá en implementar consultas de recuperación. Para clientes, se simulará una consulta de descarga de datos GPS filtrada por VIN y rango de fechas. Para sucursales, se simulará una consulta de estados por vehículo, código de falla o período de tiempo.

Comparación entre Procesamiento Histórico y Tiempo Real bajo Kappa

El procesamiento histórico es adecuado cuando se requiere analizar grandes volúmenes de datos acumulados. En el caso de ACME EV, este enfoque permite generar reportes históricos, analizar patrones de fallas, calcular indicadores operativos y preparar datos para modelos de mantenimiento predictivo. Bajo Kappa, este procesamiento se realiza mediante el reprocesamiento de eventos almacenados, no mediante una capa batch separada.

El procesamiento en tiempo real es adecuado cuando se requiere actuar rápidamente sobre eventos recientes. En ACME EV, este enfoque permite detectar fallas, niveles bajos de batería o estados anómalos casi al momento en que ocurren. Su principal ventaja es la baja latencia, aunque requiere monitoreo constante del flujo de eventos y de la infraestructura de procesamiento.

Ambos escenarios son complementarios dentro de la arquitectura Kappa. El tiempo real permite reaccionar rápidamente, mientras que el reprocesamiento histórico permite analizar el comportamiento acumulado de la flota. La ventaja principal es que ambos utilizan el mismo modelo de eventos y la misma lógica de transformación, reduciendo duplicidad e inconsistencias.

Conclusiones

La arquitectura propuesta permite a ACME EV transformar los datos generados por sus vehículos eléctricos en un activo estratégico para el negocio. Mediante una arquitectura Kappa, la empresa puede atender necesidades de monitoreo en tiempo real y análisis histórico mediante reprocesamiento de eventos, manteniendo escalabilidad, flexibilidad y simplicidad operativa.

El uso de tecnologías open source como Kafka, Spark, PostgreSQL, MongoDB y formatos como JSON, CSV y Parquet permite construir una solución viable, reproducible y adaptable al crecimiento futuro. Además, la incorporación de un Data Lake facilita la conservación de datos crudos y procesados para diferentes usos analíticos.

La seguridad y gobernanza son elementos fundamentales de la solución, ya que los datos de ubicación y estado del vehículo pueden considerarse sensibles. Por ello, el acceso debe controlarse mediante autenticación, autorización, cifrado, trazabilidad y políticas de retención claras.

Referencias

- Apache Software Foundation. (2026). *Apache Kafka Documentation*. Recuperado el 6 de mayo de 2026.
- Apache Software Foundation. (2026). *Apache Spark Documentation*. Recuperado el 6 de mayo de 2026.
- MongoDB, Inc. (2026). *MongoDB Manual*. Recuperado el 6 de mayo de 2026.
- PostgreSQL Global Development Group. (2026). *PostgreSQL Documentation*. Recuperado el 6 de mayo de 2026.
- The Apache Software Foundation. (2026). *Apache Parquet Documentation*. Recuperado el 6 de mayo de 2026.
- Kreps, J. (2014). Questioning the Lambda Architecture. O'Reilly Radar.